

第三章课后作业

专业：计算机科学与技术（拔尖人才试验班） 姓名：郭诣丰 学号：23300240002

邮箱：23300240002@m.fudan.edu.cn

我的 github: <https://github.com/23300240002> (作业已同步上传)

第三章 传输层	$k, k+1, k+2, k+3$ 中的任四个连续序号。
P3 UDP和TCP使用反码来计算校验和。假设你有下面8比特字节：01010011, 01100110, 01110100, 这些8比特字节的反码是多少？写出所有工作过程。UDP为什么要用反码的反码，而不直接使用反码呢？使用该反码方案，接收方如何检测出差错？1比特的差错可能检测不出来吗？2比特的差错呢？	b. 在t时刻，传输回发送方的报文中，ACK字段有可能值是多少？ GBN中，接收方总是发送ACK确认为其期望的序号 k ，因此均为 k 。
解：11. 01010011 + 01100110 = 10111001，无溢出 10111001 + 01110100 = 00101101，有溢出，按反码运算规则，要将高位加到最低位，= 00101110 (和) 最后的校验和为 11010001 ✓	P2. 序号空间长度为 k ，为避免图3-27的问题，对于GBN、SR协议中的每一种，允许的最大窗口大小是多少？ GBN中最大为 $k-1$ 。如果 $> k-1$ ，那么所有分组丢失，重传的旧分组序号与新分组相同，接收方无法区分是新分组还是重传分组。故 $N \leq k-1$ 。
12. 原因：直接用和，接收方计算结果可能不为0，但若使用反码和，只要结果均为1，则无差错。简化了差错检测。	SR中最大为 $k/2$ ，因为发送、接收方都维护窗口，后者可能接收性，如 $N > k/2$ ，那么两窗口可能覆盖相等的序号范围，接收方无法区分是新分组还是重传分组。
13. 将收到的所有数据块进行反码加法，若结果各位均为1，则无差错；否则有。	P4. 从主机A向B传输1字节文件，MSS=536字节，TCP序号字段4字节。
14. 1比特差错被检测到，2比特差错可能因为在不同字节和各位位置翻转(和不变)而检测不到。	a. 使TCP序号不至于用完的 L 最大值？ 4字节 = 32 b. 故空间为 2^{32} 字节， $L_{max} = 2^{32}$ 。
P22. GBN协议，发送方窗口为4，序号范围1024，在时刻t，接收方期待的下一个有序分组序号是 k ，媒介不重新排序	b. 对于在a中得到的 L ，传输此文件要用多长时间？假设传输网络，链路首部共为66字节，并加在每个报文段上，然后经15Mbps链路发送得到的分组 每个报文段数据536字节， $N = \frac{2^{32}}{536} = 8012998 \dots 308$ 总长度 $536 \times 66 = 60256$ 字节
a. 在t时刻，发送方窗口内报文序号可能是多少？ 发送方窗口包含4个连续序号，故可为 $k-3, k-2, k-1$ 。	

总字节数 = $2^{32} + 66 \times 80 = 482387164$ $\therefore t = \frac{482387164 \times 8}{155 \times 10^6} \approx 248.97 s$	P29. 在35.6节中讨论了SYN cookie. a 服务器在SYN ACK中使用特殊初始序号原因? SYN cookie使服务器在收到SYN时不分配资源,而是将连接信息编码到初始序号中。客户端返回ACK时,服务器从ACK序号解码信息并建立连接,防止攻击。
P27. 主机B已收到A的126字节, A发送两个报文段, 第一个80字节, 序号127, 第二个40字节。第一个报文段源端12302, 目的端口80.	b 攻击者得知目标使用SYN cookie, 能否直接发送ACK
a. 第一个报文段的序号、源端口、目的端口?	创建并打开连接? / 半开
序号 = $127 + 80 = 207$, 源端口302, 目的端口80.	不能。无论猜出初始序号(序号由哈希生成), 无法发送有效的ACK.
b. 如果第一个先到达, 第一个确认的确认号、源端口、目的端口?	c. 攻击者收集大量初始序号, 能否发送ACK引起全开连接
B期望207, 故确认号=207, 源端口80, 目的端口302	不能。每个初始序号与特定客户端参数绑定, 因此攻击者发送ACK时服务器无法解码(参数不匹配)
c. 如果第二个先到达, 第一个的确认号?	P33. 在35.3节中, 我们讨论了TCP的往返时间的估计, TCP避免测量重传报文段的Sample RTT, 对此你有什么看法?
B期望127, 故确认号=127	各, 重传报文段的ACK可能对应原始传输或重传, 无法确定哪个导致ACK, 从而对RTT估计的准确性, 得到偏大或偏小的值, 进而影响RTO设置。
d. 两个报文段按序到达, 第一个确认丢失, 第二个确认在超时后到达, 画时序图	P36. 在35.4节中, 我们看到TCP直到收到3个冗余ACK才执行快速重传, 你对TCP设计者没有选择在收到对报文段的一个冗余ACK后就快速重传有何看法?
① A发送: 报文段1 (seq=127, 80字节), 报文段2 (seq=207, 40字节)	
② B收到报文段1: 发送ACK (ack=207, 丢失)	
③ B收到报文段2: 发送ACK (ack=247)	
④ A超时后重传报文段1 (seq=127, 80字节)	
⑤ B收到重传报文段1: 已收到过故丢弃, 发送ACK (ack=247) ?	
⑥ A收到ACK=247 (超时后到达), 知道均被接收	

答:可以避免因网络短暂乱序而误触发重传,如果只收到一个冗余ACK,可能是报文段乱序而非丢失。连续三个冗余ACK平滑了快速重传和慢速重传风险。	为多少?
	窗口的一半, 故 $\frac{21}{2} = 10.5 \text{ MSS}$
	h. 在哪个传输轮回内发送并收到7个报文段?
	由1~6: $1+2+4+8+16=31$
P40. 考虑图3-61, 假设TCP Reno是一个经历如上所示行为的协议, 回答下列问题:	②. 7. 开始每个轮回可发2, 3...个
a. 指出TCP慢启动运行的时间间隔。	$31+32=63 < 70$, $31+32+33=96 > 70$.
$1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 16$, 从1到16, 大约间隔5个轮回。	因此共发9个传输轮回发送70个报文段。
b. 指出TCP拥塞避免运行的时间间隔。	在发送第26个传输轮回后通过收到3个冗余ACK
$32 \rightarrow 33 \rightarrow \dots \rightarrow 42$, 从16到42, 大约间隔10个轮回。	检测到有报文丢失, 拥塞窗口长度和ssthresh值应当
c. 在第16个传输轮回之后, 报文段的丢失是根据3个冗余ACK还是根据超时检测出来的?	多少?
$42 \rightarrow 21+3=24$, 因此是根据3个冗余ACK检测出来的, 进行了快速恢复。	窗口大小为8, 故 ssthresh 设为 $\frac{8}{2}=4$.
d. 在第24个传输轮回之后, 报文段的丢失是根据3个冗余ACK还是根据超时检测出来的?	cwnd 设为 $4+3=7$.
$29 \rightarrow 1$, 因此是超时检测 (骤降列)	j. 假设使用TCP Tahoe (而不是TCP Reno), 并假设在第16个传输轮回收到3个冗余ACK, 在第19个传输轮回, ssthresh 和拥塞窗口长度是什么?
e. 在第1个传输轮回里, ssthresh 的初始值设置为多少? 32 MSS . (慢启动在 $\text{cwnd} \rightarrow \text{ssthresh}$ 后线性增长)	用 Tahoe, 故直接 cwnd 置1, ssthresh 为 $\frac{\text{cwnd}}{2} = \frac{42}{2} = 21$ 故 $16 \rightarrow 17 \rightarrow 18 \rightarrow 19$ 1 2 4 8
f. 在第18个传输轮回里, ssthresh 的值设置为多少?	$21 \quad 21 \quad 21 \quad 21$
窗口的一半, 故 $\frac{42}{2} = 21 \text{ MSS}$.	第9个轮回后, ssthresh 长度为21 MSS, 窗口为8 MSS.
g. 在第24个传输轮回里, ssthresh 的值设置为多少?	k. 假设使用TCP Tahoe, 在第24个传输轮回有一个超时事件, 从第17个传输轮回开始到第24个 (包括这两个), 一共发送多少数据?
	$17 \rightarrow 18 \rightarrow 19 \rightarrow 20 \rightarrow 21 \rightarrow 22$
	cwnd $\downarrow$ 1 2 4 8 16 32 (超时)
	降列
	$\therefore 1+2+4+8+16+32=63$

P42. 在3.5.4节中, 我们讨论了在发生超时事件后将超时时间加倍. 为什么除了这种机制外, TCP还基于窗口的拥塞控制机制呢? (如3.7节中所学)

答: 超时时间加倍无法有效利用网络带宽. 而基于窗口的拥塞控制允许TCP动态调整发送速率以适应网络状况, 通过动态调整窗口大小, 实现平滑的速率控制

P47. 回想TCP吞吐量量的直观描述. 在连接速率从 $\frac{W}{RTT}$ 变到 $\frac{W}{RTT}$ 周期内, 只丢失了一个分组 (在该周期结束)

1. 证明其丢包率 (分组丢失的比率) 等于: $L = \frac{3}{8}W + \frac{3}{4}W$

证明: 拥塞避免阶段, W 以线性增长到 W , 故发送的分组总数为 $\frac{W}{2} + (\frac{W}{2} + 1) + \dots + W$

$$= (\frac{W}{2} + W) \cdot \frac{W+1}{2} = \frac{3}{2}W \cdot \frac{W+1}{2} = \frac{3W^2}{4} + \frac{3}{2}W$$

故丢包率为 $L = \frac{1}{\frac{3W^2}{4} + \frac{3}{2}W}$ (丢1个)

6. 如果一条连接丢包率为 L , 使用上面的结果, 则它的平均速率近似由下式给出: 平均速率 $\approx \frac{1.22 MSS}{RTT \sqrt{L}}$

证明: 对于大 W , 有 $L \approx \frac{8}{3W^2}$, $W \approx \sqrt{\frac{8}{3L}}$

$$\text{平均吞吐量} = \frac{W \cdot MSS}{RTT} = \frac{MSS}{RTT} \cdot \sqrt{\frac{8}{3L}} \approx \frac{1.22 MSS}{RTT \sqrt{L}}$$

P58. 我们考虑由TCP慢启动阶段引入的时延. 考虑一个客户和一个Web服务器直接连接到速率 R 的一条链路. 假定该客户要取回一个对象, 其长度正好为 $15s$ (s 是最大段长度(MSS)). 客户与服务器之间的往返时间表示为 RTT (假定为常数). 忽略协议开销, 确定在下列情况下取回该

对象的时间 (包括TCP连接创建).

- a. $\frac{4s}{R} > \frac{s}{R} + RTT > \frac{3s}{R}$
- b. $\frac{s}{R} + RTT > \frac{4s}{R}$
- c. $\frac{s}{R} > RTT$

解: ① 假设 R 很大, 此时限制为 RTT . (慢启动), 至少为 RTT (TCP建立连接) + $4RTT$ (1, 2, 4, 8) = $5RTT$

② 假设 RTT 很小, 此时限制为 R , 最快也要 $\frac{15s}{R}$

比较 $5RTT$ 和 $\frac{15s}{R}$, 即 RTT 和 $\frac{3s}{R}$

- a. $\frac{3s}{R} > RTT$, 故 $t = \frac{15s}{R}$
- b. $RTT > \frac{3s}{R}$, 故 $t = 5RTT$
- c. $\frac{s}{R} > RTT$, 故 $t = \frac{15s}{R}$